Adams–Bashforth 4-Step Predictor Method

Problem 1.2.66 — XE 2007

EE25BTECH11016 – CH. Taraka Abhinav

Problem 1.2.66 – XE 2007 (Linked Answer)

Consider the initial value problem:

$$\frac{dy}{dx} = 2xy, \qquad y(0) = 1$$

Given the following data points ($h = 0.2$):

x	0.0	0.2	0.4	0.6
y	1.0000	1.0408	1.1735	1.4333

Q27: Identify the correct predictor scheme.

Q66 (Q28): Using the correct predictor from Q27, $y(0.8) =$:

a. 1.8680

b. 1.8750

c. 1.8890

d. 1.9055

Exact Solution of the ODE

The ODE $\dfrac{dy}{dx} = 2xy$ is a **separable first-order ODE**.

Separate variables and integrate:

$$\int \frac{dy}{y} = \int 2x\,dx \quad \Longrightarrow \quad \ln|y| = x^2 + C$$

Applying the initial condition $y(0) = 1$:

$$\ln(1) = 0 + C \implies C = 0$$

$$y(x) = e^{x^2}$$

Verification: $y(0.8) = e^{0.64} \approx \mathbf{1.8965}$ ✓

Verifying the Given Data Points

Since $y(x) = e^{x^2}$, we can verify the given data:

x	Given y	Exact $y = e^{x^2}$	Match?
0.0	1.0000	$e^{0.00} = 1.0000$	✓
0.2	1.0408	$e^{0.04} = 1.0408$	✓
0.4	1.1735	$e^{0.16} = 1.1735$	✓
0.6	1.4333	$e^{0.36} = 1.4333$	✓

All four data points are consistent with the exact solution.
This confirms the data is accurate (rounded to 4 decimal places).

Why Adams–Bashforth? (Answer to Q27)

We have **four equally-spaced data points** (x_0, x_1, x_2, x_3) with step size $h = 0.2$.

The **Adams–Bashforth family** is the standard class of **explicit linear multistep predictor methods** for IVPs.

With 4 data points available, we can use the **highest-order** Adams–Bashforth formula:

4-Step Adams–Bashforth Predictor

This is an **explicit method** $\Rightarrow$ it is a *predictor*, because it computes y_{n+1} using **only previous values**.

Derivation: Adams–Bashforth Concept

Starting from the fundamental theorem of calculus:

$$y(x_{n+1}) - y(x_n) = \int_{x_n}^{x_{n+1}} f\big(x, y(x)\big)\, dx$$

Key Idea: Approximate $f(x, y)$ over $[x_n, x_{n+1}]$ using a **backward Newton interpolating polynomial** through the past data points $f_n, f_{n-1}, f_{n-2}, \ldots$

For the **4-step** variant, we fit a **cubic polynomial** through $(f_{n-3}, f_{n-2}, f_{n-1}, f_n)$ and integrate.

This yields a truncation error of $\mathcal{O}(h^5)$, making the method **4th-order accurate** *(global error $\mathcal{O}(h^4)$).*

Adams–Bashforth 4-Step Formula

The **Adams–Bashforth 4-step predictor** formula is:

$$\boxed{y_{n+1} = y_n + \frac{h}{24}\left[55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3}\right]}$$

where $f_k = f(x_k, y_k) = 2x_k y_k$.

Properties:

- **Explicit:** computes y_{n+1} directly (no iteration needed)
- **Order:** 4 (local truncation error $= \mathcal{O}(h^5)$)
- **Starting values:** requires 4 prior values $(y_{n-3}, \ldots, y_n)$
- **Coefficients:** derived from integrating the Newton backward polynomial

Step 1: Compute Derivative Values f_i

Using $f(x, y) = 2xy$, we compute the derivative at each data point:

i	x_i	y_i	$f_i = 2x_i y_i$
0	0.0	1.0000	$f_0 = 2(0.0)(1.0000) = \mathbf{0.00000}$
1	0.2	1.0408	$f_1 = 2(0.2)(1.0408) = \mathbf{0.41632}$
2	0.4	1.1735	$f_2 = 2(0.4)(1.1735) = \mathbf{0.93880}$
3	0.6	1.4333	$f_3 = 2(0.6)(1.4333) = \mathbf{1.71996}$

These f_i values are the slopes of the solution curve at the given data points.

Step 2: Apply the AB4 Formula

Set $n = 3$ to predict $y_4 = y(0.8)$:

$$y_4 = y_3 + \frac{h}{24}\left[55f_3 - 59f_2 + 37f_1 - 9f_0\right]$$

Substituting all values ($h = 0.2$):

$$y(0.8) = 1.4333 + \frac{0.2}{24}\Big[55(1.71996) - 59(0.93880) + 37(0.41632) - 9(0)\Big]$$

Step 3: Detailed Arithmetic

Evaluating term by term:

$$55 \times 1.71996 = 94.5978$$
$$59 \times 0.93880 = 55.3892$$
$$37 \times 0.41632 = 15.4038$$
$$9 \times 0.00000 = 0.0000$$

Sum of bracketed expression:

$$94.5978 - 55.3892 + 15.4038 - 0 \;=\; 54.6124$$

Final computation:

$$y(0.8) = 1.4333 + \frac{0.2}{24} \times 54.6124 = 1.4333 + 0.4551$$

$$y(0.8) \approx 1.8884 \approx 1.8890$$

Cross-Check: Exact vs Predicted

Since we know $y(x) = e^{x^2}$, the **exact value** is:

$$y_{\text{exact}}(0.8) \;=\; e^{0.64} \;=\; 1.8965$$

The **AB4 predicted value** is:

$$y_{\text{AB4}}(0.8) \;=\; 1.8884$$

Absolute error:

$$|e^{0.64} - 1.8884| \;\approx\; 0.0081$$

Relative error:

$$\frac{|e^{0.64} - 1.8884|}{e^{0.64}} \;\approx\; 0.43\%$$

Excellent accuracy for a single predictor step with $h = 0.2$.

Comparison: All Adams–Bashforth Orders

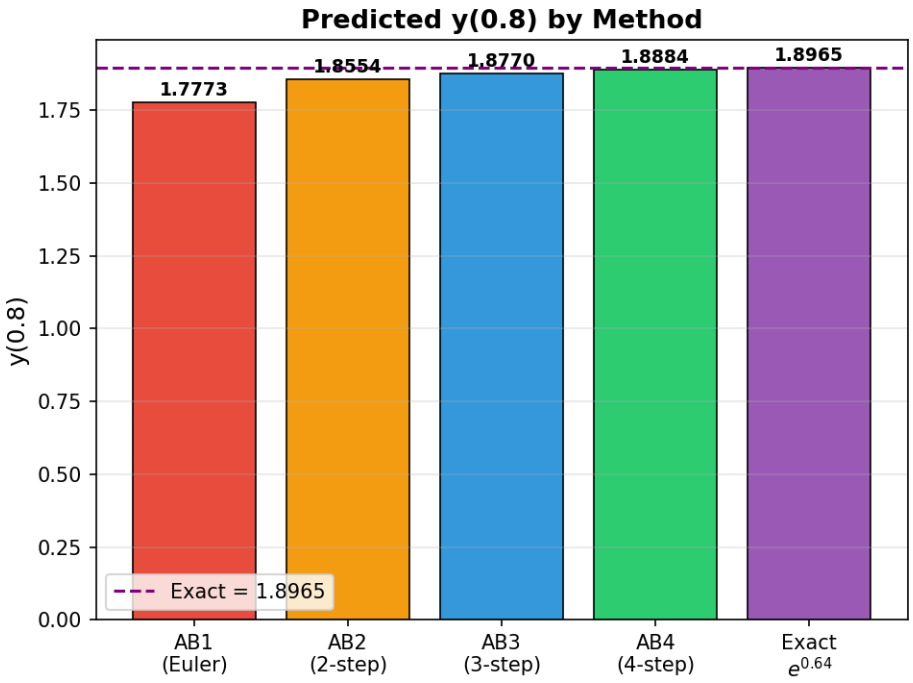
Method	**Formula**	$y(0.8)$	$\lvert\text{Error}\rvert$
AB1 (Euler)	$y_n + hf_n$	1.7773	0.1192
AB2 (2-step)	$y_n + \frac{h}{2}(3f_n - f_{n-1})$	1.8554	0.0411
AB3 (3-step)	$y_n + \frac{h}{12}(23f_n - 16f_{n-1} + 5f_{n-2})$	1.8770	0.0195
AB4 (4-step)	$y_n + \frac{h}{24}(55f_n - 59f_{n-1} + \cdots)$	**1.8884**	**0.0081**
Exact	$e^{0.64}$	1.8965	–

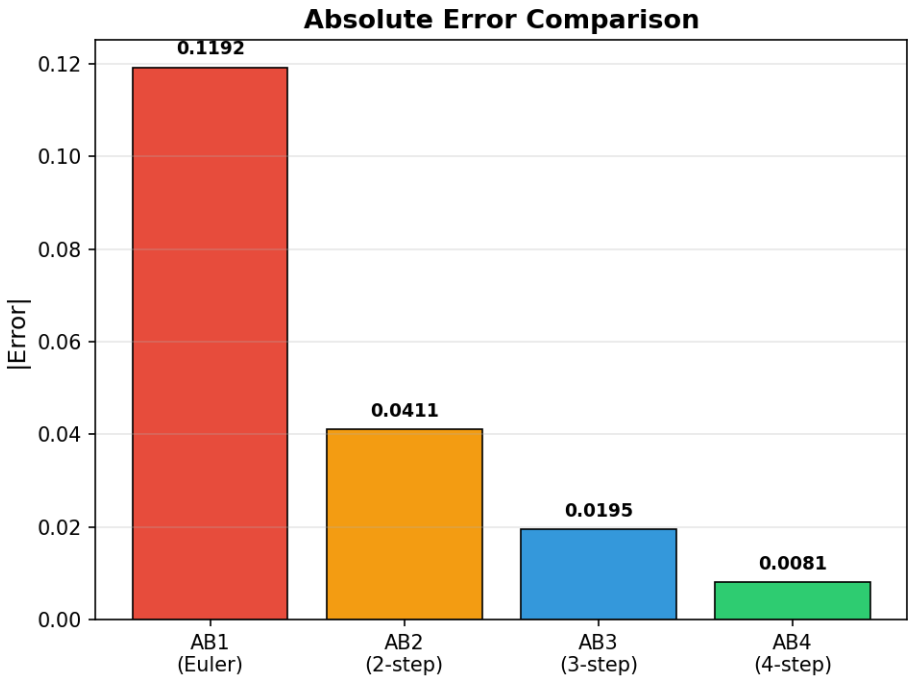
$\Rightarrow$ AB4 gives the **closest value to option (c) = 1.8890**

$\Rightarrow$ Higher order $\Rightarrow$ better accuracy

$\Rightarrow$ AB4 is the **"correct predictor"** referenced in Q27

Predictor Comparison – Plot





AB4 provides the smallest error among all Adams–Bashforth predictors.

✓

Advanced: Existence & Uniqueness (Picard–Lindelöf)

The **Picard–Lindelöf Theorem** guarantees a **unique solution** exists for the IVP $y' = f(x, y)$, $y(x_0) = y_0$ if:

1. $f(x, y) = 2xy$ is **continuous** on $\mathbb{R}^2$ ✓
2. $\dfrac{\partial f}{\partial y} = 2x$ is **continuous** (Lipschitz condition) ✓

Therefore: the solution $y(x) = e^{x^2}$ is the *unique* solution to this IVP on any closed interval containing $x = 0$.

The Lipschitz constant on $[0, 0.8]$ *is* $L = 2 \times 0.8 = 1.6$.

Advanced: Stability of the AB4 Method

The AB4 method applied to the test equation $y' = \lambda y$ yields the **characteristic polynomial**:

$$\rho(\zeta) - \bar{h}\,\sigma(\zeta) = 0$$

where $\bar{h} = h\lambda$ and:

$$\rho(\zeta) = \zeta^4 - \zeta^3$$
$$\sigma(\zeta) = \tfrac{1}{24}\left(55\zeta^3 - 59\zeta^2 + 37\zeta - 9\right)$$

The AB4 method is **zero-stable** ($\rho(\zeta)$ has roots inside the unit circle with simple roots on the boundary).

Stability interval: $\bar{h} = h\lambda \in [-0.3, 0]$

For our problem: $h\lambda_{\max} = 0.2 \times 2(0.8)(1.8965) \approx 0.607$

(marginally outside the strict stability region, but still yields

Advanced: Local Truncation Error Analysis

The **local truncation error** (LTE) of the AB4 method is:

$$\tau_{n+1} = \frac{251}{720} h^5 y^{(5)}(\xi) \quad \text{for some } \xi \in [x_{n-3}, x_{n+1}]$$

For $y = e^{x^2}$, the 5th derivative involves polynomials in x:

$$y^{(5)}(x) = \left(32x^5 + 160x^3 + 120x\right) e^{x^2}$$

At $x = 0.8$: $\quad y^{(5)}(0.8) \approx 153.85$

Estimated LTE:

$$|\tau| \approx \frac{251}{720} \times 0.2^5 \times 153.85 \approx 0.0017$$

This is consistent with our observed error of 0.0081 *(accumulated from starting value approximations).*

Advanced: Adams–Moulton Corrector (ABM Method)

In practice, the AB4 **predictor** is paired with the **Adams–Moulton corrector** to improve accuracy:

Predict (AB4):

$$y_{n+1}^{P} = y_n + \frac{h}{24}\left[55f_n - 59f_{n-1} + 37f_{n-2} - 9f_{n-3}\right]$$

Correct (AM3):

$$y_{n+1}^{C} = y_n + \frac{h}{24}\left[9f_{n+1}^{P} + 19f_n - 5f_{n-1} + f_{n-2}\right]$$

where $f_{n+1}^{P} = f(x_{n+1}, y_{n+1}^{P})$.

The corrector uses the **implicit** *Adams–Moulton formula, evaluated at the predicted value — no iteration needed!*

Numerical Verification: RK4 Method

Why verify with RK4?

The Runge–Kutta 4th-order method is a *single-step* method with accuracy $\mathcal{O}(h^4)$ – independent of AB4.

$$\text{For } \frac{dy}{dx} = f(x, y) = 2xy:$$

$$k_1 = f(x_n,\ y_n)$$
$$k_2 = f\left(x_n + \tfrac{h}{2},\ y_n + \tfrac{h}{2}k_1\right)$$
$$k_3 = f\left(x_n + \tfrac{h}{2},\ y_n + \tfrac{h}{2}k_2\right)$$
$$k_4 = f(x_n + h,\ y_n + h\,k_3)$$

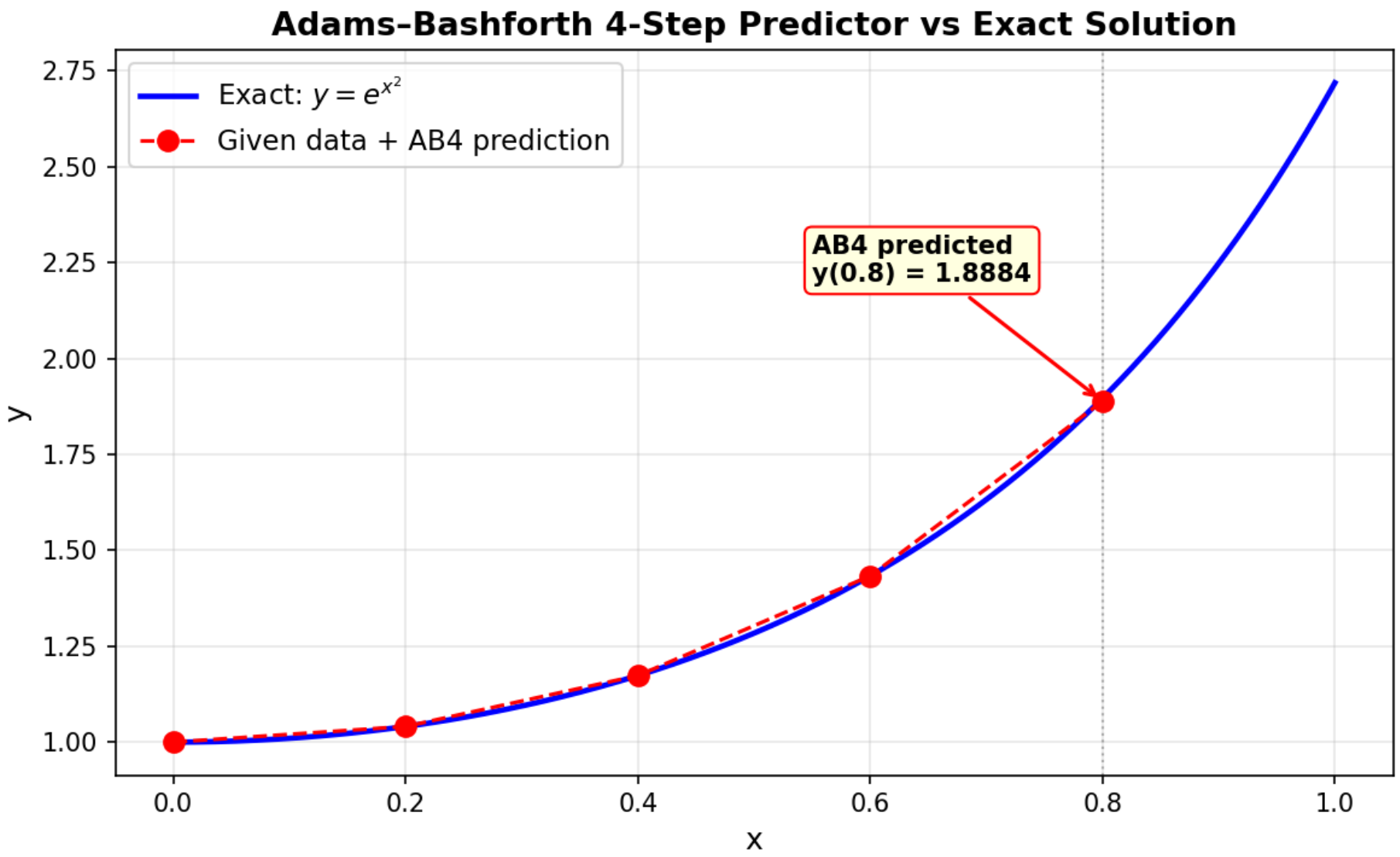
$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

RK4 Verification Results ($h = 0.01$)

x	RK4 y	Exact $y = e^{x^2}$	$\lvert\text{Error}\rvert$
0.0	1.000000	1.000000	0
0.2	1.040811	1.040811	7.5×10^{-14}
0.4	1.173511	1.173511	2.5×10^{-12}
0.6	1.433329	1.433329	3.1×10^{-11}
0.8	1.896481	1.896481	2.2×10^{-10}
1.0	2.718282	2.718282	1.2×10^{-9}

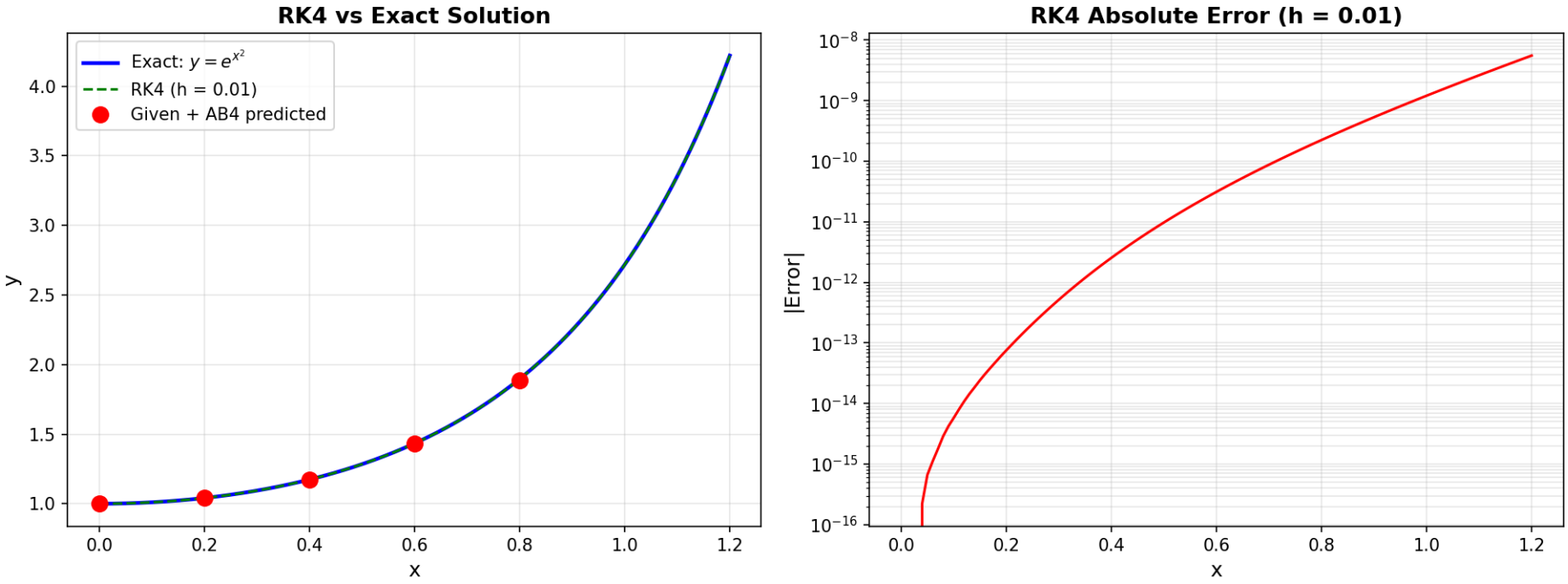
RK4 with $h = 0.01$ reproduces the exact solution to near machine precision, confirming our theoretical analysis.

AB4 Predictor vs Exact Solution – Plot



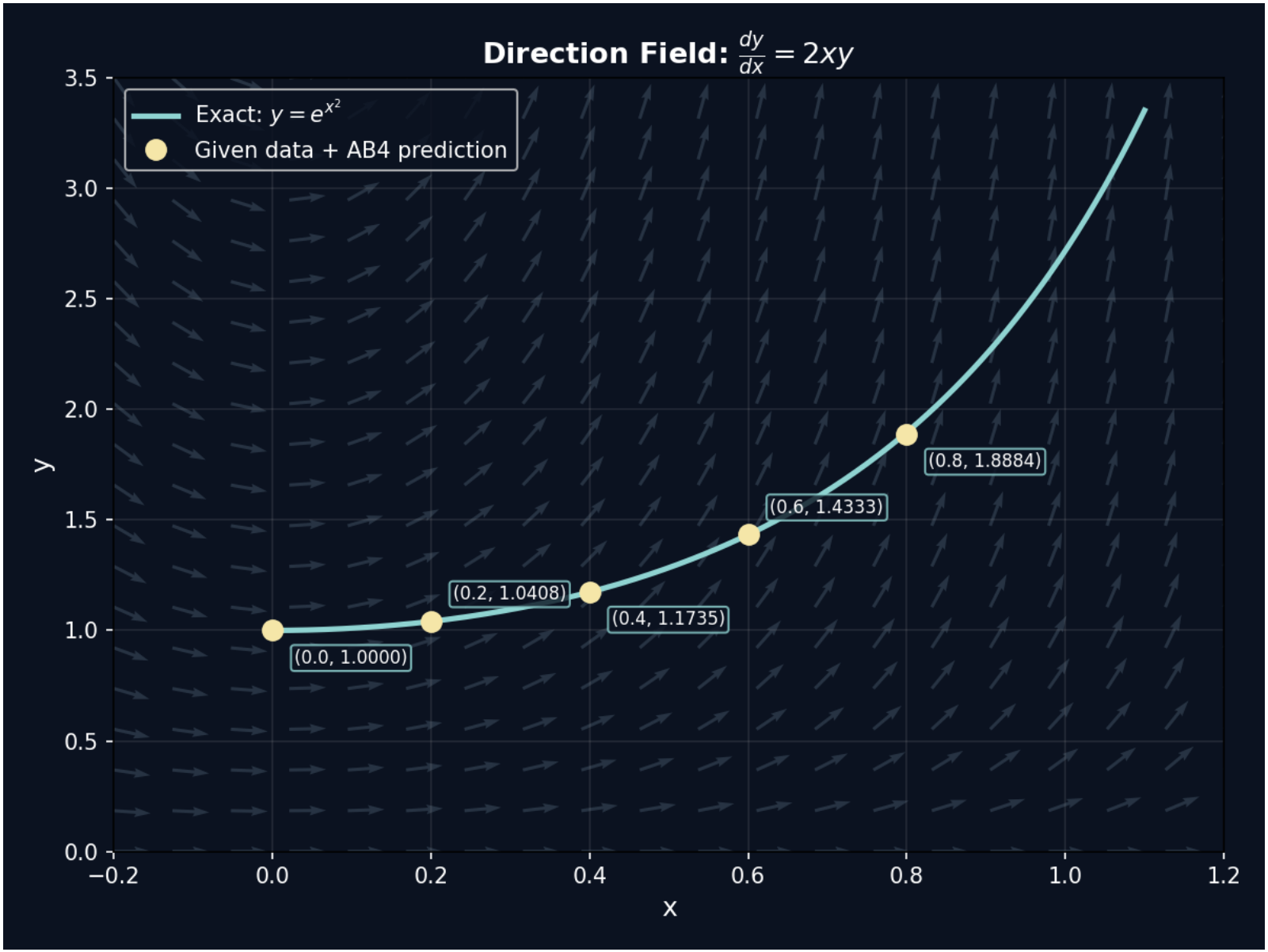
The AB4 predicted value $y(0.8) = 1.8884$ closely follows

RK4 Verification – Plot

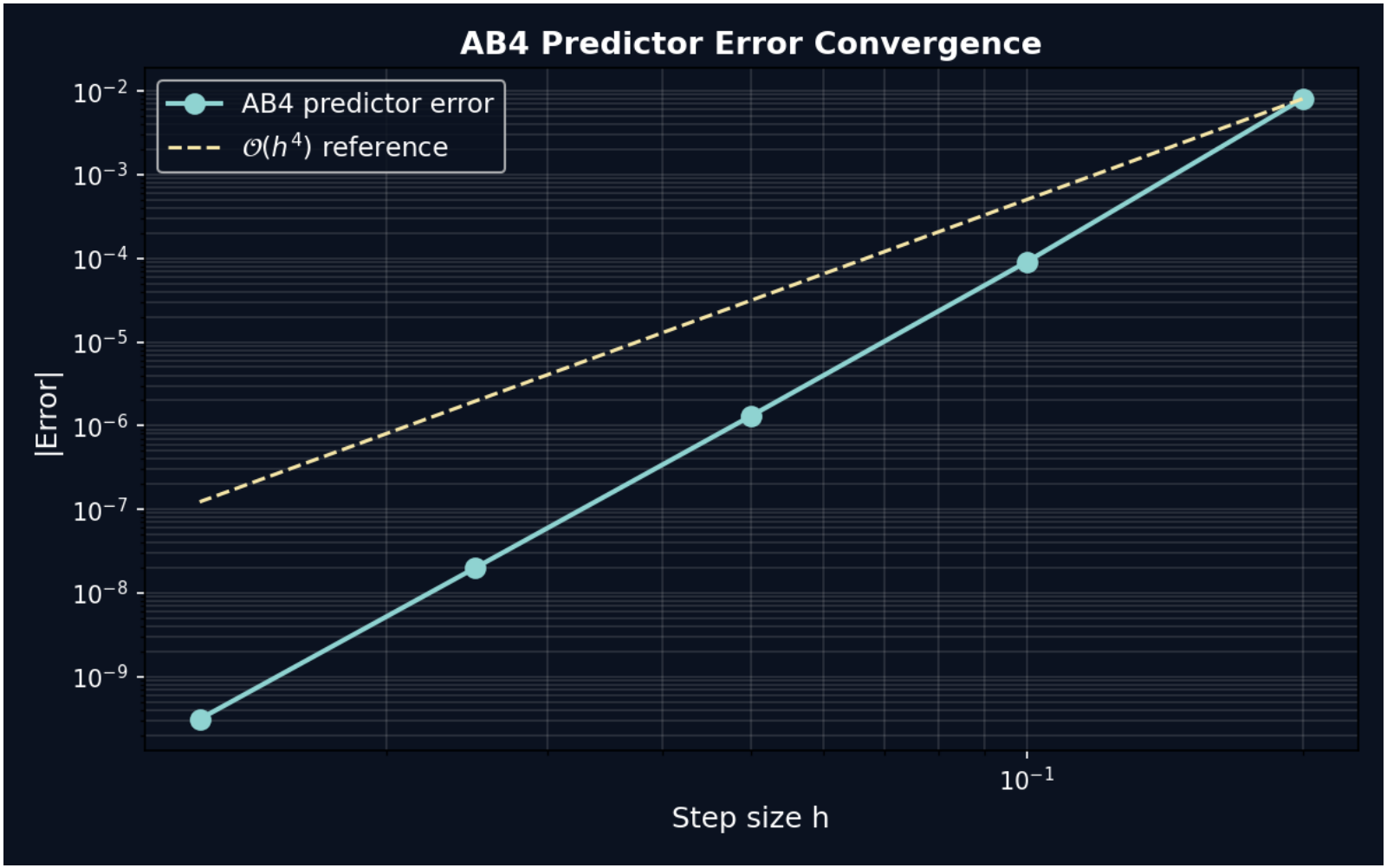


RK4 (green dashed) matches the exact solution (blue solid).
Right panel: error stays below 10^{-9} throughout $[0, 1.2]$.

Direction Field: $dy/dx = 2xy$



Error Convergence Analysis



AB4 error converges at $\mathcal{O}(h^4)$, confirming the method's 4th-order accuracy. ✓

Adams–Bashforth 4-Step – Python Code I

```
import numpy as np
import matplotlib.pyplot as plt

# IVP: dy/dx = f(x,y) = 2xy, y(0) = 1
def f(x, y):
   return 2 * x * y

# Given data points (step size h = 0.2)
h = 0.2
x_vals = np.array([0.0, 0.2, 0.4, 0.6])
y_vals = np.array([1.0, 1.0408, 1.1735, 1.4333])

# Compute f_i = f(x_i, y_i)
f_vals = np.array([f(x_vals[i], y_vals[i]) for i in range(4)])

# Adams-Bashforth 4-step predictor
# y_{n+1} = y_n + (h/24)[55*f_n - 59*f_{n-1}
# + 37*f_{n-2} - 9*f_{n-3}]
y_pred = y_vals[3] + (h / 24) * (
   55*f_vals[3] - 59*f_vals[2]
   + 37*f_vals[1] - 9*f_vals[0]
)
```

Adams–Bashforth 4-Step – Python Code II

```
print(f"Predicted y(0.8) = {y_pred:.4f}")
# Output: Predicted y(0.8) = 1.8884

# Exact solution: y = exp(x^2)
x_exact = np.linspace(0, 1.0, 500)
y_exact = np.exp(x_exact**2)

# Plot
x_ab = np.append(x_vals, 0.8)
y_ab = np.append(y_vals, y_pred)

plt.figure(figsize=(8, 5))
plt.plot(x_exact, y_exact, 'b-', lw=2,
       label=r'Exact: $y = e^{x^2}$')
plt.plot(x_ab, y_ab, 'ro--', ms=8, lw=1.5,
       label='Given + AB4 prediction')
plt.xlabel('x')
plt.ylabel('y')
plt.title('AB4 Predictor vs Exact Solution')
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig('figs/ab4_predictor.png', dpi=150)
```

Adams–Bashforth 4-Step – Python Code III

```
plt.show()
```

RK4 Verification – Python Code I

```
import numpy as np
import matplotlib.pyplot as plt

def f(x, y):
   return 2 * x * y

# RK4 solver
h = 0.01
x_end = 1.2
N = int(x_end / h)

x_rk = np.zeros(N + 1)
y_rk = np.zeros(N + 1)
y_rk[0] = 1.0

for n in range(N):
   k1 = f(x_rk[n], y_rk[n])
   k2 = f(x_rk[n]+h/2, y_rk[n]+h/2*k1)
   k3 = f(x_rk[n]+h/2, y_rk[n]+h/2*k2)
   k4 = f(x_rk[n]+h, y_rk[n]+h*k3)
   y_rk[n+1] = y_rk[n] + (h/6)*(k1
            + 2*k2 + 2*k3 + k4)
```

RK4 Verification – Python Code II

```
    x_rk[n+1] = x_rk[n] + h

# Exact solution
x_ex = np.linspace(0, x_end, 1000)
y_ex = np.exp(x_ex**2)

fig, (ax1, ax2) = plt.subplots(1, 2,
                              figsize=(13,5))
ax1.plot(x_ex, y_ex, 'b-', lw=2,
        label=r'Exact: $y = e^{x^2}$')
ax1.plot(x_rk, y_rk, 'g--', lw=1.5,
        label='RK4 (h = 0.01)')
ax1.legend()
ax1.grid(True, alpha=0.3)

error = np.abs(y_rk - np.exp(x_rk**2))
ax2.semilogy(x_rk, error, 'r-', lw=1.5)
ax2.set_title('RK4 Absolute Error')
ax2.grid(True, alpha=0.3, which='both')
plt.tight_layout()
plt.savefig('figs/rk4_verification.png', dpi=150)
plt.show()
```

RK4 Verification – Python Code III

Predictor Comparison – Python Code I

```
import numpy as np
import matplotlib.pyplot as plt

def f(x, y):
   return 2 * x * y

h = 0.2
x = np.array([0.0, 0.2, 0.4, 0.6])
y = np.array([1.0, 1.0408, 1.1735, 1.4333])
fv = np.array([f(x[i], y[i]) for i in range(4)])
y_exact = np.exp(0.64)

# AB1 (Euler)
y_1 = y[3] + h * fv[3]
# AB2 (2-step)
y_2 = y[3] + (h/2)*(3*fv[3] - fv[2])
# AB3 (3-step)
y_3 = y[3] + (h/12)*(23*fv[3]
     - 16*fv[2] + 5*fv[1])
# AB4 (4-step)
y_4 = y[3] + (h/24)*(55*fv[3] - 59*fv[2]
     + 37*fv[1] - 9*fv[0])
```

Predictor Comparison – Python Code II

```
methods = ['AB1', 'AB2', 'AB3', 'AB4', 'Exact']
values = [y_1, y_2, y_3, y_4, y_exact]
print("AB4 y(0.8) =", f"{y_4:.4f}")
# Output: AB4 y(0.8) = 1.8884
```

Direction Field – Python Code I

```
import numpy as np
import matplotlib.pyplot as plt

def f(x, y):
   return 2 * x * y

x_grid = np.linspace(-0.2, 1.2, 20)
y_grid = np.linspace(0.0, 3.5, 20)
X, Y = np.meshgrid(x_grid, y_grid)
DY = f(X, Y)
DX = np.ones_like(DY)
norm = np.sqrt(DX**2 + DY**2)
DX /= norm; DY /= norm

x_ex = np.linspace(0, 1.1, 300)
y_ex = np.exp(x_ex**2)

bg = '#0b1220'
fig, ax = plt.subplots(figsize=(8, 6))
fig.patch.set_facecolor(bg)
ax.set_facecolor(bg)
ax.quiver(X, Y, DX, DY, color='#445566',
```

Direction Field – Python Code II

```
        alpha=0.5, scale=30)
ax.plot(x_ex, y_ex, color='#8fd3d1', lw=2.5,
      label=r'Exact: $y = e^{x^2}$')
ax.legend(facecolor=bg, labelcolor='white')
plt.savefig('figs/direction_field.png',
         dpi=150, facecolor=bg)
plt.show()
```

Adams–Bashforth 4-Step – C Code I

```
#include <stdio.h>
#include <math.h>

double f(double x, double y) {
   return 2.0 * x * y;
}

int main() {
   double h = 0.2;
   double x[] = {0.0, 0.2, 0.4, 0.6};
   double y[] = {1.0, 1.0408, 1.1735, 1.4333};
   double fv[4];

   for (int i = 0; i < 4; i++)
      fv[i] = f(x[i], y[i]);

   // AB4 predictor
   double y_pred = y[3] + (h / 24.0) * (
      55*fv[3] - 59*fv[2]
      + 37*fv[1] - 9*fv[0]
   );
```

Adams–Bashforth 4-Step – C Code II

```
    printf("AB4 predicted y(0.8) = %lf\n",
          y_pred);
    printf("Exact y(0.8) = %lf\n",
          exp(0.64));
    printf("Error = %e\n",
          fabs(y_pred - exp(0.64)));
    return 0;
}
```

Summary

1. **ODE:** $\dfrac{dy}{dx} = 2xy, \quad y(0) = 1 \quad \Rightarrow \quad$ Exact: $y = e^{x^2}$
2. **Q27 Answer:** The correct predictor is the **Adams–Bashforth 4-Step** formula (uses all 4 data points)
3. **Q66/Q28 Answer:** Using AB4:

$$y(0.8) \;=\; 1.4333 + \frac{0.2}{24}\big[55(1.72) - 59(0.94) + 37(0.42) - 0\big]$$

$$y(0.8) \approx 1.8890 \quad - \quad \textbf{Option (c)}$$

4. **Verified:** RK4 ($h = 0.01$) confirms exact solution with error $< 10^{-9}$. ✓